Test Coverage Report

Guideline reference: §6.4 (test coverage requirements), §7.2 (Unit Tests stage generates a coverage report), §8.2 (quality gates), §10.1 item 6.

- **Report date:** 2026-10-27
- **Source:** coverage reports produced by the CI pipelines, not measured by hand.
- Backend: Backend CI run 40631877653 on the `feature/DDP-63-backend-branch-coverage` branch (DDP-#63), artifacts `coverage-<service>` (`coverage.xml`). This is the first run with branch coverage and production-code-only measurement; re-export it from `develop` once DDP-#63 is merged.
- Frontend: Frontend CI run 40475255079 (commit `574e385`), artifact `coverage-report` (`lcov.info`).
- **Test inventory:** see `test-register.md` (every test case) and `endpoint-register.md` (every API endpoint).

Targets (§6.4, §8.2)

Layer	Required	Measured by
Unit tests, all new code	80% (lines / branches)	pytest-cov, Jest
API endpoints	100% have at least one integration test	`endpoint-register.md`
Critical business logic	90% branch coverage	defined in the SDS

Backend (pytest-cov, production code, lines and branches)

Measured on production code only. The test files live inside each service folder and the Alembic migrations are scaffolding, so they are excluded (`[tool.coverage.run] omit` in `edura-backend/pyproject.toml`, DDP-#63). Earlier reports counted them, which inflated line coverage (tests always run) and diluted branch coverage (migration branches).

Service	Lines covered	Lines total	Line coverage	Branches covered	Branches total	Branch coverage	Test cases
admin_service	320	327	97.9%	32	32	100.0%	66
analytics_service	668	677	98.7%	113	114	99.1%	97
assessment_service	1,352	1,363	99.2%	268	276	97.1%	141
auth_service	626	636	98.4%	117	120	97.5%	88
content_service	194	197	98.5%	38	38	100.0%	37
course_service	849	857	99.1%	147	152	96.7%	81
enrollment_service	584	591	98.8%	140	144	97.2%	65

Service	Lines covered	Lines total	Line coverage	Branches covered	Branches total	Branch coverage	Test cases
notification_service	246	257	95.7%	29	30	96.7%	38
payment_service	412	416	99.0%	37	38	97.4%	44
progress_service	439	449	97.8%	80	82	97.6%	36
user_service	563	571	98.6%	106	106	100.0%	85
Total	**6,253**	**6,341**	**98.6%**	**1,107**	**1,132**	**97.8%**	**778**

Every service is above the 80% target for lines and for branches.

Critical business logic (§6.4: 90% branch coverage). `auth_service` reaches **97.5%** and `payment_service` **97.4%** (37 of 38 branches), so the 90% target is met. An earlier measurement showed 87.9% for `payment_service`; that figure counted migration scaffolding and test files, not payment logic. The uncovered branches are few: for example one in `analytics_service`, one in `notification_service`, one in `payment_service` (start-up code), and 8 in `assessment_service`.

Frontend (Jest)

Metric	Covered	Total	Coverage
Lines	7,187	7,315	98.3%
Branches	4,078	4,455	91.5%
Functions	2,097	2,161	97.0%

The figures cover production code only: Jest's `collectCoverageFrom` already excludes the test files. The frontend has 854 test cases in 31 test files. Lines, branches and functions are all above 80%, and branches (91.5%) are above the 90% branch target.

Other suites

Suite	Test cases	What it checks
`edura-tests/integration`	5	Payment and enrolment flow, against an in-process mock gateway
`edura-tests/smoke`	1	End-to-end critical path on staging, run after each staging deploy
`edura-infra`	12	Nginx configuration and RabbitMQ topology
Total, all suites	**1,650**	

How coverage is enforced in CI

Repository	Gate	Where
edura-backend	Coverage on changed lines of a pull request must be at least 80% (`diff-cover`); the report is uploaded as an artifact	`ci.yml`, job `verify-service`
edura-frontend	Overall coverage must be at least 80% (Jest `coverageThreshold`); changed lines of a pull request must be at least 80% (`diff-cover`)	`jest.config.js`, `ci.yml` (DDP-#58, pending merge when this report was written)

Test results

All test jobs in the two runs linked above passed (every `Verify <service>` job in the backend run and the Unit Tests job in the frontend run). The test register's `Status` column stays blank because it is generated from source; the execution record is the CI run history.

Gaps

#	Gap	Suggested action
1	Backend figures come from the DDP-#63 branch, not yet from `develop`	Merge DDP-#63, then re-export this report from `develop`
2	No performance test results (guideline §10.1 item 9) and no load tests in the repositories	Run a load test against staging and add `performance-report`
3	The report is a snapshot of one CI run	Re-export after the last merge before submission
4	Report is Markdown; the deliverable list asks for HTML or PDF	Export to PDF as `coverage-sprint8.pdf` for the final submission